

```
In[*]:= << mGRG`Stensor`
```

Chap. 3 Symmetries, Operations, and Rules

3.1 Tensors and Their Symmetries

3.1.1 Indices

반대칭 텐서 F_{ab} 를 정의한다. Covariant (또는 lower) 인덱스를 입력하기 위해 인덱스의 이름 앞에 'l'(ell)을 덧붙인다:

```
In[*]:= Tdefine[MaxwellF, "-ba", PrintAs -> "F"]
```

```
In[*]:= MaxwellF[la, lb]
```

Out[*]=

F_{ab}

Contravariant (또는 upper) 인덱스를 입력하려면 인덱스의 이름 앞에 'u'를 덧붙인다:

```
In[*]:= MaxwellF[ua, ub]
```

Out[*]=

F^{ab}

인덱스 이름으로 사용되도록 미리 준비된 (lower and upper)인덱스들은 다음과 같다:

```
In[*]:= GetIndices[Latin]
```

Out[*]=

{la, lb, lc, ld, le, lf, lg, lh, li, lj, lk, ll, lm, ln, lo, lp, lq,
lr, ls, lt, lu, lv, lw, lx, ly, lz, ua, ub, uc, ud, ue, uf, ug, uh,
ui, uj, uk, ul, um, un, uo, up, uq, ur, us, ut, uu, uv, uw, ux, uy, uz}

```
In[*]:= {MaxwellF[lc, ld], MaxwellF[uc, ud]}
```

Out[*]=

{ F_{cd} , F^{cd} }

3.1.2 Some Common Tensors

계량 텐서 g_{ab} , 리만 텐서 $R_{abc}{}^d$, 리치 텐서 R_{ab} , 스칼라 곡률 R 등이 미리 정의되어 있다:

```
In[*]:= {MaxwellF[la, lb], Metricg[la, lb],  
RiemannCD[la, lb, lc, ud], RicciCD[la, lb], ScalarCD[]}
```

Out[*]=

{ F_{ab} , g_{ab} , $R_{abc}{}^d$, R_{ab} , R }

3.1.3 Symmetries

전자기장 텐서 F_{ab} 는 반대칭이고, 리치 텐서 R_{ab} 는 대칭이다:

```
In[*]:= {MaxwellF[lb, la], RicciCD[lb, la]}
% // TindexSort
```

```
Out[*]= {Fba, Rba}
```

```
Out[*]= {-Fab, Rab}
```

리만 텐서의 대칭은 좀 더 복잡하다: $R_{abcd} = R_{[ab][cd]} = R_{cdab}$:

```
In[*]:= {RiemannCD[lb, la, lc, ld],
RiemannCD[la, lb, ld, lc], RiemannCD[lc, ld, la, lb]}
% // TindexSort
```

```
Out[*]= {Rbacd, Rabdc, Rcdab}
```

```
Out[*]= {-Rabcd, -Rabcd, Rabcd}
```

프로그램 내부적으로 텐서 대칭은 ‘부호를 갖는 순열’로 표현한다. 여기서 ‘부호를 갖는 순열’의 데이터 구조는 {순열의 Cycle 표기, 대칭 또는 반대칭(+1 또는 -1)}이다:

```
In[*]:= GetSymmetry[MaxwellF]
GetSymmetry[RicciCD]
```

```
Out[*]= GenSet[{Cycles[{{1, 2}}], -1}]
```

```
Out[*]= GenSet[{Cycles[{{1, 2}}], 1}]
```

사용자 입장에서는 텐서 대칭을 인덱스 표기법으로 입력 또는 출력한다. 예를 들면, 반대칭인 전자기장의 대칭은 “-ba”, 대칭인 리치 텐서의 대칭은 “+ba”로 입력한다.

{Cycles[...], ±1} 표기법을 인덱스 표기법으로 바꿀 수 있다:

```
In[*]:= GetSymmetry[RiemannCD]
```

```
Out[*]= GenSet[{Cycles[{{1, 2}}], -1},
{Cycles[{{3, 4}}], -1}, {Cycles[{{1, 3}, {2, 4}}], 1}]
```

```
In[*]:= % // GStoString
```

```
Out[*]= -bacd-abdc+cdab
```

텐서 대칭은 유한군을 형성한다. 따라서 군론을 이용하여 모든 가능한 대칭을 구할 수 있다:

```
In[*]:= % // AllPermutations
Out[*]=
+abcd-abdc-bacd+badc+cdab-cdba-dcab+dcba
```

3.1.4 Defining Your Own Tensors

이름이 'anti'이고 a로 출력되는 랭크 2인 반대칭 텐서를 정의한다:

```
In[*]:= Tdefine[anti, "-ba", PrintAs -> "a"]
```

```
In[*]:= anti[lb, la]
% // TindexSort
```

```
Out[*]=
aba
```

```
Out[*]=
-aab
```

텐서 대칭의 내부 표현을 얻는다:

```
In[*]:= GetSymmetry[anti]
```

```
Out[*]=
GenSet[{Cycles[{{1, 2}}], -1}]
```

인덱스 교환 표기법으로 바꾼다:

```
In[*]:= % // GStoString
```

```
Out[*]=
-ba
```

텐서 anti의 모든 가능한 대칭을 구한다:

```
In[*]:= % // AllPermutations
```

```
Out[*]=
+ab-ba
```

텐서를 정의할 때 대칭이 모순되면 오류 메시지를 출력한다:

```
In[*]:= Tdefine[wrong, "ba-ba", PrintAs -> "w"]
```

```
Out[*]=
General: ba-ba is not a valid perm; it has inconsistent weights.
```

```
Out[*]=
$Failed
```

오류가 발생하면 그 텐서는 당연히 정의되지 않는다:

```
In[*]:= wrong[la, lb]
```

```
Out[*]:= wrong[la, lb]
```

기존 텐서의 대칭을 갖는 새로운 텐서를 정의할 수 있다. 리만 텐서의 대칭을 갖는 텐서를 정의해 보자.

리만 텐서 대칭의 내부 표현을 얻는다:

```
In[*]:= GetSymmetry[RiemannCD]
```

```
Out[*]:= GenSet[{Cycles[{{1, 2}}], -1},
  {Cycles[{{3, 4}}], -1}, {Cycles[{{1, 3}, {2, 4}}], 1}]
```

인덱스 표기법으로 바꾼다:

```
In[*]:= permW = % // GStoString
```

```
Out[*]:= -bacd-abdc+cdab
```

이름이 'newRiemann', 출력은 r, 대칭은 리만 텐서의 대칭을 갖는 새로운 텐서를 정의한다:

```
In[*]:= Tdefine[newRiemann, permW, PrintAs -> "r"]
```

```
In[*]:= newRiemann[ld, lc, lb, la]
% // TindexSort
```

```
Out[*]:= rdcba
```

```
Out[*]:= rabcd
```

대칭이 없는 랭크 2와 랭크 1인 텐서를 정의한다:

```
In[*]:= Tdefine[w, 2]; Tdefine[v, 1]
```

```
In[*]:= {w[lb, la], w[uc, la], v[la], v[ua]}
```

```
Out[*]:= {wba, wca, va, va}
```

랭크 2인 대칭 텐서를 정의한다:

```
In[*]:= Tdefine[q, "ba"]
```

```
In[*]:= q[lb, la]
% // TindexSort
```

```
Out[*]=
```

 q_{ba}

```
Out[*]=
```

 q_{ab}

랭크 2인 반대칭 텐서 p와 t를 정의한다:

```
In[*]:= Tdefine[p, "-ba"]; Tdefine[t, "-ba"]
```

```
In[*]:= {p[lb, la], t[lb, la]}
% // TindexSort
```

```
Out[*]=
```

 $\{p_{ba}, t_{ba}\}$

```
Out[*]=
```

 $\{-p_{ab}, -t_{ab}\}$

이름이 cyc, 출력이 c, 그리고 순환 대칭을 갖는 텐서를 정의한다:

```
In[*]:= Tdefine[cyc, "cab", PrintAs -> "c"]
```

```
In[*]:= {cyc[lc, la, lb], cyc[la, lb, lc], cyc[lb, lc, la]}
% // TindexSort
```

```
Out[*]=
```

 $\{c_{cab}, c_{abc}, c_{bca}\}$

```
Out[*]=
```

 $\{c_{abc}, c_{abc}, c_{abc}\}$

The weights in the permutations should be **ALWAYS** +1 or -1. (붉은 색은 MathTensor와의 차이점을 표시한다.)

3.2 Contraction and the Metric Tensor

3.2.1 Forming New Tensors by Contraction

```
In[*]:= Tdefine[anti, "-ba", PrintAs -> "a"]
```

```
In[*]:= anti[la, lb] × anti[ua, lc]
% + anti[lb, ld] × anti[lc, ud]
```

```
Out[*]=
```

$$a_{ab} a^a_c$$

```
Out[*]=
```

$$a_{ba} a^a_c + a_{ab} a^a_c$$

임시로 프로그램의 자동적 조정 기능을 끄자:

```
In[*]:= Off[AutoFlag]
```

축약된(Contracted) 인덱스를 (내부 표현의 직접적인 출력이 보기에 좋지 않은) 더미 인덱스로 바꾸는 명령은 Dum이다:

```
In[*]:= %% // Dum
```

```
Out[*]=
```

$$a_{b\text{Latin\$10699}} a^{\text{Latin\$10699}}_c + a_{\text{Latin\$10702}b} a^{\text{Latin\$10702}}_c$$

더미 인덱스를 (출력에 보기 좋은) 인덱스로 재명명한다:

```
In[*]:= % // ResetDummies
```

```
Out[*]=
```

$$a_{ba} a^a_c + a_{ab} a^a_c$$

AutoFlag이 'On'이면 ResetDummies를 포함한 몇 가지 연산이 자동적으로 적용된다:

```
In[*]:= On[AutoFlag] (*default*)
```

```
In[*]:= %% // TindexSort
```

```
Out[*]=
```

$$2 a_{ab} a^a_c$$

NB: Dum does **NOT** have a second argument that can be used to specify the number at which dummy index numbering in an expression should be begin.

3.2.2 Raising and Lowering Indices with the Metric Tensor

이름이 Metricg이고 출력이 g인 계량 텐서가 기본적으로 정의되어 있다:

```
In[*]:= Metricg[ua, ub] × anti[lb, lc]
```

```
Out[*]=
```

$$a_{bc} g^{ab}$$

계량 텐서를 이용하여 축약된 인덱스를 올린다:

```
In[*]:= % // Absorbg
```

```
Out[*]=
```

$$a^a_c$$

계량 텐서 Metricg는 대칭 텐서이다:

```
In[*]:= Metricg[lb, la]
```

```
% // TindexSort
```

```
Out[*]=
```

$$g_{ba}$$

```
Out[*]=
```

$$g_{ab}$$

계량 텐서의 인덱스가 (up, down) 또는 (down, up)이면 크로네커 델타이다:

```
In[*]:= Metricg[la, ub]
```

```
Out[*]=
```

$$\delta_a^b$$

크로네커 델타를 통한 축약된 인덱스의 재조정은 자동적으로 적용된다:

```
In[*]:= Kdelta[la, ub] × RicciCD[lb, lc]
```

```
Out[*]=
```

$$R_{ac}$$

계량 텐서를 이용하여 인덱스를 재조정하려면 Absorbg 명령을 사용한다:

```
In[*]:= Metricg[ua, ub] × Metricg[lb, lc]
```

```
% // Absorbg
```

```
Out[*]=
```

$$g_{bc} g^{ab}$$

```
Out[*]=
```

$$\delta^a_c$$

리치 텐서:

```
In[*]:= Metricg[uc, ud] × RiemannCD[la, lc, lb, ld]
```

```
% // Absorbg
```

```
Out[*]=
```

$$g^{cd} R_{acbd}$$

```
Out[*]=
```

$$R_{ab}$$

스칼라 곡률:

```
In[*]:= Metricg[ua, ub] × RicciCD[la, lb]
% // Absorbg
```

```
Out[*]=
```

$$g^{ab} R_{ab}$$

```
Out[*]=
```

$$R$$

3.2.3 Preventing Automatic Absorption of the Metric

MathTensor에서는 모든 텐서 표현에 Absorbg가 자동적으로 적용되지만, mGRG에서는 명시적으로 적용시켜야 한다. 단, 크로네커 델타를 통한 축약된 인덱스의 재조정은 기본적으로 자동 적용된다.

```
In[*]:= Metricg[ua, ub] × anti[lb, lc]
% // Absorbg
```

```
Out[*]=
```

$$a_{bc} g^{ab}$$

```
Out[*]=
```

$$a^a{}_c$$

```
In[*]:= Metricg[la, lb] (RicciCD[ub, uc] + anti[ub, uc])
% // Absorbg
```

```
Out[*]=
```

$$a^{bc} g_{ab} + g_{ab} R^{bc}$$

```
Out[*]=
```

$$a_a{}^c + R_a{}^c$$

Absorbg[expr]은 Absorb[expr, Metricg]와 같다:

```
In[*]:= Tdefine[someG, "ba", PrintAs → "G"]
```

```
In[*]:= someG[la, lb]
RicciCD[li, lj] × someG[ui, uj]
Absorb[%, someG]
```

```
Out[*]=
```

$$G_{ab}$$

```
Out[*]=
```

$$R_{ab} G^{ab}$$

```
Out[*]=
```

$$R$$

크로네커 델타를 통한 축약된 인덱스의 자동 조정을 일시로 끄자:

```
In[*]:= Off[KdeltaFlag]
```



```
In[*]:= Kdelta[la, ub] × someG[ua, lb]
Out[*]=
```

$$\delta_a^b G^a_b$$

크로네커 델타를 통한 축약된 인덱스의 자동 조정은 기본이다:

```
In[*]:= On[KdeltaFlag]
```

```
In[*]:= Kdelta[la, ub] × someG[ua, lb]
Out[*]=
```

$$G^a_a$$

3.3 Simplification

3.3.1 Methods of Simplifying Tensor Expressions

더미 인덱스의 이름은 중요하지 않으므로 (ResetDummies 함수를 이용하여) 자동적으로 재명명된다:

```
In[*]:= expr1 = RicciCD[ua, ub] × RiemannCD[la, li, lb, lj] +
          RicciCD[ua, ub] × RiemannCD[la, lj, lb, li]
```

```
Out[*]=
```

$$R^{ab} R_{aibj} + R^{ab} R_{ajbi}$$

```
In[*]:= expr2 = RiemannCD[la, lb, lc, ld] × RiemannCD[ua, ub, ue, uf] ×
          RiemannCD[uc, ud, le, lf] + RiemannCD[lc, ld, ug, uh] ×
          RiemannCD[le, lf, lg, lh] × RiemannCD[uc, ud, ue, uf]
```

```
Out[*]=
```

$$R_{ab}^{cd} R_{efcd} R^{abef} + R_{abcd} R^{abef} R_{ef}^{cd}$$

```
In[*]:= expr3 = RicciCD[la, lc] × RicciCD[lb, uc] × RicciCD[ua, ub] +
          RicciCD[la, lb] × RicciCD[ua, lc] × RicciCD[ub, uc]
```

```
Out[*]=
```

$$R_{ab} R_c^b R^{ac} + R_{ab} R_c^a R^{bc}$$

```
In[*]:= expr4 = RiemannCD[ua, ub, uc, ud] × RiemannCD[la, lb, lc, ld] +
          RiemannCD[ua, uc, ub, ud] × RiemannCD[la, lc, lb, ld]
```

```
Out[*]=
```

$$2 R_{abcd} R^{abcd}$$

다만 더미 인덱스의 이름을 재명명하는 것만으로는 텐서 표현을 단순화(simplification)할 수 없다. 텐서의 대칭도 고려해야 한다:

```
In[*]:= expr1
Out[*]=
```

$$R^{ab} R_{aibj} + R^{ab} R_{ajbi}$$

3.3.2 Simplifying with Symmetries

```
In[*]:= Tdefine[anti, "-ba", PrintAs -> "a"]
이름이 'sym'이고 출력은 s인 대칭 텐서를 정의한다:
```

```
In[*]:= Tdefine[sym, "ba", PrintAs -> "s"]
반대칭 텐서와 대칭 텐서를 축약하면 0이 된다:
```

```
In[*]:= anti[la, lb] × sym[ua, ub]
% // Tsimplify
```

```
Out[*]=
```

$$a_{ab} s^{ab}$$

```
Out[*]=
```

$$0$$

반대칭 텐서의 trace는 0이다.

```
In[*]:= anti[la, ua]
% // Tsimplify
```

```
Out[*]=
```

$$a_a^a$$

```
Out[*]=
```

$$0$$

Tsimplify는 텐서 대칭을 고려하여 단순화한다:

```
In[*]:= anti[lb, ld] × sym[lc, ub] - anti[ld, ub] × sym[lb, lc]
% // Tsimplify
```

```
Out[*]=
```

$$-a_d^a s_{ac} + a_{ad} s_c^a$$

```
Out[*]=
```

$$-2 a_{da} s_c^a$$

```
In[*]:= expr = m anti[lb, ld] × sym[lc, ub] + n anti[ld, ub] × sym[lb, lc]
% // Tsimplify
% // Factor
```

```
Out[*]=
```

$$n a_d^a s_{ac} + m a_{ad} s_c^a$$

```
Out[*]=
```

$$-m a_{da} s_c^a + n a_{da} s_c^a$$

```
Out[*]=
```

$$(-m + n) a_{da} s_c^a$$

MathTensor의 Tsimplify는 패턴 매칭 방법을 사용하고, mGRG는 ‘Butler-Portugal’ 알고리즘에 기반한 정규화(canonicalization) 방법을 사용하는 Mathematica 내장 함수 TensorReduce를 사용한다. 그런데 패턴 매칭 방법을 이용한 단순화는 실질적으로 사용하기에 곤란할 정도로 매우 비효율적이다. 따라서 mGRG에서 텐서 표현의 단순화는 정규화 방법을 사용한다.

```
In[*]:= expr = expr3 + expr4
% // Tsimplify
```

```
Out[*]=
```

$$R_{ab} R_c^b R^{ac} + R_{ab} R_c^a R^{bc} + 2 R_{abcd} R^{abcd}$$

```
Out[*]=
```

$$2 R_{ab} R_c^a R^{bc} + 2 R_{abcd} R^{abcd}$$

```
In[*]:= expr2
% // Tsimplify
```

```
Out[*]=
```

$$R_{ab}^{cd} R_{efcd} R^{abef} + R_{abcd} R^{abef} R_{ef}^{cd}$$

```
Out[*]=
```

$$2 R_{abcd} R_{ef}^{ab} R^{cdef}$$

3.3.3 Simplifying Traces and Contractions of Products

필요한 텐서들을 정의하자:

```
In[*]:= Tdefine[nosym1, 2, PrintAs → "ns1"];
Tdefine[nosym2, 2, PrintAs → "ns2"];
Tdefine[nosym3, 2, PrintAs → "ns3"];
Tdefine[n, 4]
```

아래의 두 항은 더미 인덱스의 이름만 제외하면 동일하므로 자동 조정된다:

```
In[*]:= expr = nosym1[la, ub] × nosym2[lb, uc] × nosym3[lc, ua] +
nosym3[la, ub] × nosym1[lb, uc] × nosym2[lc, ua]
```

```
Out[*]=
```

$$2 ns1_a^b ns2_b^c ns3_c^a$$

아래 표현에 Tsimplify를 적용해 보자:

```
In[*]:= expr =
n[ua, ub, lc, ld] × n[uc, ud, le, lh] × n[ue, uf, lg, lf] × n[ug, uh, la, lb] -
n[ua, ub, lc, ld] × n[uc, ud, le, lf] × n[ue, uf, lg, lb] × n[ug, uh, la, lh]
```

```
Out[*]=

$$-n_{cd}^{ab} n_{ef}^{cd} n_{gb}^{ef} n_{ah}^{gh} + n_{cd}^{ab} n_{ef}^{cd} n_{hg}^{eg} n_{ab}^{hf}$$

```

```
In[*]:= Timing[Tsimplify[expr]]
```

```
Out[*]=
{0.017195, 0}
```

3.3.4 Canonical Ordering of Indices

텐서 표현의 단순화는 Tsimplify 명령 하나만 사용해도 충분하다.

```
In[*]:= expr2
% // Tsimplify
```

```
Out[*]=

$$R_{ab}^{cd} R_{efcd} R^{abef} + R_{abcd} R^{abef} R_{ef}^{cd}$$

```

```
Out[*]=

$$2 R_{abcd} R_{ef}^{ab} R^{cdef}$$

```

```
In[*]:= expr3
% // Tsimplify
```

```
Out[*]=

$$R_{ab} R_c^b R^{ac} + R_{ab} R_c^a R^{bc}$$

```

```
Out[*]=

$$2 R_{ab} R_c^a R^{bc}$$

```

3.4 Operations on Tensors

3.4.1 Symmetrization and Antisymmetrization

리만 텐서의 첫 번째와 세 번째 인덱스를 대칭화와 반대칭화한다:

```
In[*]:= RiemannCD[la, lb, lc, ld]
SymmetrizeIndices[%, {la, lc}]
AntisymmetrizeIndices[%%, {la, lc}]
```

```
Out[*]=
```

$$R_{abcd}$$

```
Out[*]=
```

$$\frac{1}{2} R_{abcd} + \frac{1}{2} R_{cbad}$$

```
Out[*]=
```

$$\frac{1}{2} R_{abcd} - \frac{1}{2} R_{cbad}$$

리만 텐서의 처음 두 인덱스는 반대칭이므로 대칭화하면 0이 된다:

```
In[*]:= SymmetrizeIndices[RiemannCD[la, lb, lc, ld], {la, lb}]
% // TindexSort
```

```
Out[*]=
```

$$\frac{1}{2} R_{abcd} + \frac{1}{2} R_{bacd}$$

```
Out[*]=
```

$$0$$

```
In[*]:= AntisymmetrizeIndices[RiemannCD[la, lb, lc, ld], {la, lb}]
% // TindexSort
```

```
Out[*]=
```

$$\frac{1}{2} R_{abcd} - \frac{1}{2} R_{bacd}$$

```
Out[*]=
```

$$R_{abcd}$$

반대칭 텐서 MaxwellF를 정의하고, 인덱스 a, b, c를 반대칭화한다:

```
In[*]:= Tdefine[MaxwellF, "-ba", PrintAs -> "F"]
```

```
In[*]:= AntisymmetrizeIndices[MaxwellF[la, lb] × RicciCD[lc, ld], {la, lb, lc}]
% // TindexSort
```

```
Out[*]=
```

$$\frac{1}{6} F_{bc} R_{ad} - \frac{1}{6} F_{cb} R_{ad} - \frac{1}{6} F_{ac} R_{bd} + \frac{1}{6} F_{ca} R_{bd} + \frac{1}{6} F_{ab} R_{cd} - \frac{1}{6} F_{ba} R_{cd}$$

```
Out[*]=
```

$$\frac{1}{3} F_{bc} R_{ad} - \frac{1}{3} F_{ac} R_{bd} + \frac{1}{3} F_{ab} R_{cd}$$

NB: **No** PairSymmetrize[] and PairAntisymmetrize[].

랭크-4 텐서를 정의한 후, 완전 대칭(totally symmetric)으로 설정한다:

```
In[*]:= Tdefine[tens, 4];
SetSymmetry[tens, "4+"]
```

```
In[*]:= tens[ld, lc, lb, la] // TindexSort
```

```
Out[*]:=
```

```
tensabcd
```

물론 텐서를 정의하면서 인덱스 대칭을 지정할 수 있다:

```
In[*]:= Tdefine[tens, "4+"]
tens[ld, lc, lb, la] // TindexSort
```

```
Out[*]:=
```

```
tensabcd
```

텐서의 대칭을 완전 반대칭(totally anti-symmetric)으로 설정한다:

```
In[*]:= SetSymmetry[tens, "4-"]
tens[ld, lb, lc, la] // TindexSort
```

```
Out[*]:=
```

```
-tensabcd
```

3.4.2 Differential Form Operations

외미분(Exterior Derivative)

2-form 을 정의한다:

```
In[*]:= Fdefine[w2, 2]
```

2-form w2의 외미분인 3-form dw2를 IndexedTensor 표현으로 바꾼다:

```
In[*]:= XD[w2[]]
FtoC[%, {la, lb, lc}]
```

```
Out[*]:=
```

```
dw2
```

```
Out[*]:=
```

```
 $\nabla_a w_{2bc} - \nabla_b w_{2ac} + \nabla_c w_{2ab}$ 
```

공변 도함수 ∇_a 를 보통 도함수 ∂_a 로 바꾼다:

```
In[*]:= % // CDtoBD
% // Tsimplify
```

```
Out[*]:=
```

```
 $\partial_a w_{2bc} - \partial_b w_{2ac} + \partial_c w_{2ab} + \Gamma_{bc}^d w_{2ad} - \Gamma_{cb}^d w_{2ad} - \Gamma_{ac}^d w_{2bd} - \Gamma_{ca}^d w_{2db} - \Gamma_{ab}^d w_{2dc} + \Gamma_{ba}^d w_{2dc}$ 
```

```
Out[*]:=
```

```
 $\partial_a w_{2bc} - \partial_b w_{2ac} + \partial_c w_{2ab}$ 
```

외미분을 IndexedTensor 표현으로 바꿀 때 공변 도함수 대신 보통 도함수를 사용할 수 있게 한다:

In[*]:= Off[XDtoCDfrag]

In[*]:= FtoC[XD[w2[]], {la, lb, lc}]

Out[*]=

$$\partial_a w_{bc} - \partial_b w_{ac} + \partial_c w_{ab}$$

In[*]:= On[XDtoCDfrag] (* default *)

1-form과 3-form을 정의한다:

In[*]:= Fdefine[w1, 1]; Fdefine[w3, 3]

공간을 3차원으로 설정한다:

In[*]:= SetDimension[3]

3-form의 외미분은 4-form이고, 3차원 공간에서는 0이다.

In[*]:= XD[w3[]]

Out[*]=

0

외적(Exterior Product)

1-form과 2-form의 외적은 3-form이다:

In[*]:= w1 ^ w2
% // XD

Out[*]=

w1 ^ w2

Out[*]=

0

1-form을 정의한다:

In[*]:= Fdefine[s1, 1]

In[*]:= XP[s1, w1]
% // XD

Out[*]=

s1 ^ w1

Out[*]=

-s1 ^ dw1 + w1 ^ ds1

HodgeStar

3차원에서 w1의 Hodge dual은 2-form이다:

```
In[*]:= HodgeStar[w1]
FtoC[%, {la, lb}]
```

```
Out[*]=
```

```
*w1
```

```
Out[*]=
```

$$\in^c_{ab} w1_c$$

좌표 표현

Indexed 0-Form을 정의한다:

```
In[*]:= Fdefine[x[ua], 0]
```

성분값을 지정한다:

```
In[*]:= SetComponents[x[ua], {r Sin[θ] Cos[φ], r Sin[θ] Sin[φ], r Cos[θ]}]
{x[1], x[2], x[3]}
```

```
Out[*]=
```

```
{r Cos[φ] Sin[θ], r Sin[θ] Sin[φ], r Cos[θ]}
```

구형 좌표계에 대한 외미분 규칙:

```
In[*]:= xdRule = {XD[Sin[θ_]] => Cos[θ] XD[θ], XD[Cos[θ_]] => -Sin[θ] XD[θ]}
```

```
Out[*]=
```

```
{dSin[θ_] => Cos[θ] dθ, dCos[θ_] => -Sin[θ] dθ}
```

```
In[*]:= dx1 = XD[x[1]] /. xdRule
dx2 = XD[x[2]] /. xdRule
dx3 = XD[x[3]] /. xdRule
```

```
Out[*]=
```

```
Cos[φ] Sin[θ] dr + r Cos[θ] Cos[φ] dθ - r Sin[θ] Sin[φ] dφ
```

```
Out[*]=
```

```
Sin[θ] Sin[φ] dr + r Cos[θ] Sin[φ] dθ + r Cos[φ] Sin[θ] dφ
```

```
Out[*]=
```

```
Cos[θ] dr - r Sin[θ] dθ
```

x^3 방향에 수직인 면적 요소:

```
In[*]:= dArea3 = XP[dx1, dx2] // Simplify
```

```
Out[*]=
```

```
r Sin[θ]^2 dr^dφ + r^2 Cos[θ] Sin[θ] dθ^dφ
```

체적 요소:


```
In[*]:= dVolume = XP[dx1, dx2, dx3]
% // Simplify
```

```
Out[*]=
```

$$r^2 \cos[\theta]^2 \cos[\phi]^2 \sin[\theta] \, \mathbf{dr} \wedge \mathbf{d\theta} \wedge \mathbf{d\phi} + r^2 \cos[\phi]^2 \sin[\theta]^3 \, \mathbf{dr} \wedge \mathbf{d\theta} \wedge \mathbf{d\phi} + r^2 \cos[\theta]^2 \sin[\theta] \sin[\phi]^2 \, \mathbf{dr} \wedge \mathbf{d\theta} \wedge \mathbf{d\phi} + r^2 \sin[\theta]^3 \sin[\phi]^2 \, \mathbf{dr} \wedge \mathbf{d\theta} \wedge \mathbf{d\phi}$$

```
Out[*]=
```

$$r^2 \sin[\theta] \, \mathbf{dr} \wedge \mathbf{d\theta} \wedge \mathbf{d\phi}$$

내적(Interior Product)

1-form, 2-form, Indexed 0-form을 정의한다:

```
In[*]:= Fdefine[w1, 1]; Fdefine[w2, 2]; Fdefine[x[ua], 0]
```

구형 좌표계를 설정한다:

```
In[*]:= SetComponents[x[ua], {r, \theta, \phi}]
```

w2의 좌표 표현을 얻는다:

```
In[*]:= 1/2 FtoC[w2[], {la, lb}] \times XP[XD[x[ua]], XD[x[ub]]]
% // SumDum // TindexSort
```

```
Out[*]=
```

$$\frac{1}{2} w_{2ab} \, \mathbf{dx}^a \wedge \mathbf{dx}^b$$

```
Out[*]=
```

$$w_{212} \, \mathbf{dr} \wedge \mathbf{d\theta} + w_{213} \, \mathbf{dr} \wedge \mathbf{d\phi} + w_{223} \, \mathbf{d\theta} \wedge \mathbf{d\phi}$$

```
In[*]:= CoordRep[w2, {r, \theta, \phi}]
```

```
Out[*]=
```

$$w_{212} \, \mathbf{dr} \wedge \mathbf{d\theta} + w_{213} \, \mathbf{dr} \wedge \mathbf{d\phi} + w_{223} \, \mathbf{d\theta} \wedge \mathbf{d\phi}$$

dw1의 좌표 표현을 얻는다:

```
In[*]:=
1
- FtoC[XD[w1[]], {la, lb}] × XP[XD[x[ua]], XD[x[ub]]]
2
% // SumDum
% // TindexSort
% // CollectForm
```

Out[*]=

$$\nabla_a w_{1b} \, dx^a \wedge dx^b$$

Out[*]=

$$-\nabla_2 w_{11} \, dr \wedge d\theta + \nabla_1 w_{12} \, dr \wedge d\theta - \nabla_3 w_{11} \, dr \wedge d\phi + \nabla_1 w_{13} \, dr \wedge d\phi - \nabla_3 w_{12} \, d\theta \wedge d\phi + \nabla_2 w_{13} \, d\theta \wedge d\phi$$

Out[*]=

$$-\nabla_2 w_{11} \, dr \wedge d\theta + \nabla_1 w_{12} \, dr \wedge d\theta - \nabla_3 w_{11} \, dr \wedge d\phi + \nabla_1 w_{13} \, dr \wedge d\phi - \nabla_3 w_{12} \, d\theta \wedge d\phi + \nabla_2 w_{13} \, d\theta \wedge d\phi$$

Out[*]=

$$\left(-\nabla_2 w_{11} + \nabla_1 w_{12}\right) dr \wedge d\theta + \left(-\nabla_3 w_{11} + \nabla_1 w_{13}\right) dr \wedge d\phi + \left(-\nabla_3 w_{12} + \nabla_2 w_{13}\right) d\theta \wedge d\phi$$

```
In[*]:= CoordRep[XD[w1], {r, \theta, \phi}]
```

Out[*]=

$$\left(-\nabla_2 w_{11} + \nabla_1 w_{12}\right) dr \wedge d\theta + \left(-\nabla_3 w_{11} + \nabla_1 w_{13}\right) dr \wedge d\phi + \left(-\nabla_3 w_{12} + \nabla_2 w_{13}\right) d\theta \wedge d\phi$$

w1과 dw1의 내적에 대한 좌표 표현:

```
In[*]:= Off[XDtoCDfrag]
```

```
In[*]:= IP[w1, XD[w1]]
```

Out[*]=

$$\mathcal{L}_{w1} dw1$$

```
In[*]:= FtoC[%, {la}] × XD[x[ua]]
```

```
% // SumDum
% // TindexSort
% // CollectForm
```

Out[*]=

$$-\partial_a w_{1b} w_1^b \, dx^a + \partial_a w_{1b} w_1^a \, dx^b$$

Out[*]=

$$\partial_2 w_{11} w_1^2 \, dr - \partial_1 w_{12} w_1^2 \, dr + \partial_3 w_{11} w_1^3 \, dr - \partial_1 w_{13} w_1^3 \, dr - \partial_2 w_{11} w_1^1 \, d\theta + \partial_1 w_{12} w_1^1 \, d\theta + \partial_3 w_{12} w_1^3 \, d\theta - \partial_2 w_{13} w_1^3 \, d\theta - \partial_3 w_{11} w_1^1 \, d\phi + \partial_1 w_{13} w_1^1 \, d\phi - \partial_3 w_{12} w_1^2 \, d\phi + \partial_2 w_{13} w_1^2 \, d\phi$$

Out[*]=

$$\partial_2 w_{11} w_1^2 \, dr - \partial_1 w_{12} w_1^2 \, dr + \partial_3 w_{11} w_1^3 \, dr - \partial_1 w_{13} w_1^3 \, dr - \partial_2 w_{11} w_1^1 \, d\theta + \partial_1 w_{12} w_1^1 \, d\theta + \partial_3 w_{12} w_1^3 \, d\theta - \partial_2 w_{13} w_1^3 \, d\theta - \partial_3 w_{11} w_1^1 \, d\phi + \partial_1 w_{13} w_1^1 \, d\phi - \partial_3 w_{12} w_1^2 \, d\phi + \partial_2 w_{13} w_1^2 \, d\phi$$

Out[*]=

$$\begin{aligned} & \left(\partial_2 w_{11} w_1^2 - \partial_1 w_{12} w_1^2 + \partial_3 w_{11} w_1^3 - \partial_1 w_{13} w_1^3\right) dr + \\ & \left(-\partial_2 w_{11} w_1^1 + \partial_1 w_{12} w_1^1 + \partial_3 w_{12} w_1^3 - \partial_2 w_{13} w_1^3\right) d\theta + \\ & \left(-\partial_3 w_{11} w_1^1 + \partial_1 w_{13} w_1^1 - \partial_3 w_{12} w_1^2 + \partial_2 w_{13} w_1^2\right) d\phi \end{aligned}$$

```
In[*]:= CoordRep[IP[w1, XD[w1]], {r, θ, ϕ}]
```

```
Out[*]=
```

$$\begin{aligned} & (\partial_2 w_{11} w_1^2 - \partial_1 w_{12} w_1^2 + \partial_3 w_{11} w_1^3 - \partial_1 w_{13} w_1^3) \mathbf{dr} + \\ & (-\partial_2 w_{11} w_1^1 + \partial_1 w_{12} w_1^1 + \partial_3 w_{12} w_1^3 - \partial_2 w_{13} w_1^3) \mathbf{d\theta} + \\ & (-\partial_3 w_{11} w_1^1 + \partial_1 w_{13} w_1^1 - \partial_3 w_{12} w_1^2 + \partial_2 w_{13} w_1^2) \mathbf{d\phi} \end{aligned}$$

```
In[*]:= On[XDtoCDfrag] (* default *)
```

NB: No MatrixMap, and MatrixXP.

DegreeForm, ZeroDegreeQ

Indexed 2-Form을 정의한다:

```
In[*]:= Fdefine[h[ua], 2];
h[ua]
```

```
Out[*]=
```

h^a

```
In[*]:= {DegreeForm[h[ua]], ZeroDegreeQ[h[ua]]}
```

```
Out[*]=
```

{2, False}

```
In[*]:= FtoC[h[ua], {lb, lc}]
```

```
Out[*]=
```

h_{bc}^a

2-form h^a 를 IndexedTensor로 변환하였으므로 h_{bc}^a 는 0-form이다:

```
In[*]:= {DegreeForm[h[lb, lc, ua]], ZeroDegreeQ[h[lb, lc, ua]]}
```

```
Out[*]=
```

{0, True}

3.4.3 Levi-Civita Epsilon Tensor

Levi-Civita 텐서는 시공간의 차원과 동일한 수의 인덱스를 갖는 완전 반대칭 텐서이다.

시공간을 4차원으로 설정한다:

```
In[*]:= SetDimension[4]
```

```
In[*]:= Epsilon[lb, la, lc, ld]
% // TindexSort
```

```
Out[*]=
```

ϵ_{bacd}

```
Out[*]=
```

$-\epsilon_{abcd}$

Levi-Civita 텐서의 성분 ϵ_{1234} 의 값을 지정한다. (나머지 성분은 완전 반대칭을 이용하여 자동으로 설정된다):

```
In[*]:= SetComponents[Epsilon[-1, -2, -3, -4], Sqrt[Abs[Detg]]]
Epsilon[-1, -2, -3, -4]

Out[*]=

$$\sqrt{\text{Abs}[\text{Detg}]}$$

```

p -form에 대한 Dual 연산 (참고. R. M. Wald 문제 2-(a)):

$$(*\alpha)_{b_1 \dots b_{n-p}} = \frac{1}{p!} \alpha^{a_1 \dots a_p} \epsilon_{a_1 \dots a_p b_1 \dots b_{n-p}}$$

$$**\alpha = (-1)^{s+p(n-p)} \alpha$$

여기서 s 는 계량 텐서의 ‘signature’ (number of negative eigenvalues of metric)이다.

```
In[*]:= Tdefine[MaxwellF, "-ba", PrintAs -> "F"]

In[*]:= DualStar[MaxwellF[la, lb], {uc, ud}]

Out[*]=

$$\frac{1}{2} \epsilon^{abcd} F_{ab}$$


In[*]:= aRule = RuleUnique[starF[c_, d_],  $\frac{1}{2}$  Epsilon[ua, ub, c, d]  $\times$  MaxwellF[la, lb]]

Out[*]=

$$\text{starF}[c\_ , d\_ ] \mapsto \text{DumFresh}\left[\frac{1}{2} \epsilon_{cd}^{\text{Latin\$12590Latin\$12592}} F_{\text{Latin\$12590Latin\$12592}}\right]$$


In[*]:= starF[ua, ub] /. aRule
starF[la, lb] /. aRule

Out[*]=

$$\frac{1}{2} \epsilon^{cdab} F_{cd}$$


Out[*]=

$$\frac{1}{2} \epsilon_{ab}^{cd} F_{cd}$$

```

시공간의 metric signature (‘number of minus’)를 설정한다:

```
In[*]:= SetSig[1] (* (-1,1,1,1) *)

In[*]:= DualStar[starF[la, lb] /. aRule, {uc, ud}]

Out[*]=

$$\frac{1}{4} \epsilon^{abcd} \epsilon^{ef}_{ab} F_{ef}$$

```

EpsilonProductRule[] (참고. R. M. Wald, Appendix B, Eq. (B.2.13)):

$$\epsilon^{a_1 \dots a_j a_{j+1} \dots a_n} \epsilon_{a_1 \dots a_j b_{j+1} \dots b_n} = (-1)^s j! (n-j)! \delta_{b_{j+1}}^{[a_{j+1}} \dots \delta_{b_n}^{a_n]}$$

```
In[*]:= % /. EpsilonProductRule[]
% // Absorbg
% // TindexSort
```

Out[*]=

$$\frac{1}{2} F_{ab} g^{cb} g^{da} - \frac{1}{2} F_{ab} g^{ca} g^{db}$$

Out[*]=

$$-\frac{1}{2} F^{cd} + \frac{F^{dc}}{2}$$

Out[*]=

$$-F^{cd}$$

시공간을 3차원 유클리드 공간의 특성들로 설정한다:

```
In[*]:= SetDimension[3]; SetSig[0]; Detg = 1;
```

```
In[*]:= Tdefine[s, 1]; Tdefine[v, 1]; Tdefine[w, 1]
```

```
In[*]:= Epsilon[ua, lb, lc] × s[ub] × v[uc]
```

Out[*]=

$$\epsilon^a_{bc} s^b v^c$$

```
In[*]:= sRule = RuleUnique[sxv[a_], Epsilon[a, lb, lc] × s[ub] × v[uc]]
```

Out[*]=

$$sxv[a_] \mapsto \text{DumFresh}[\epsilon_{a\text{Latin\$12690Latin\$12692}} s^{\text{Latin\$12690}} v^{\text{Latin\$12692}}]$$

벡터 외적에 대한 공식:

$$(\vec{s} \times \vec{v}) \times \vec{w} = (\vec{s} \cdot \vec{w}) \vec{v} - (\vec{v} \cdot \vec{w}) \vec{s}$$

```
In[*]:= Epsilon[ua, lb, lc] (sxv[ub] /. sRule) w[uc]
% /. EpsilonProductRule[]
% // Absorbg
```

Out[*]=

$$\epsilon^a_{bc} \epsilon^b_{de} s^d v^e w^c$$

Out[*]=

$$g_{bc} s^c v^a w^b - g_{bc} s^a v^c w^b$$

Out[*]=

$$-s^a v_b w^b + s_b v^a w^b$$

3.4.4 Derivative Operators

```
In[*]:= Tdefine[MaxwellF, "-ba", PrintAs → "F"]
```

```
In[*]:= expr = RicciCD[la, lb] × MaxwellF[ub, lc] + RicciCD[la, lc]
```

```
Out[*]:=
```

$$F^b{}_c R_{ab} + R_{ac}$$

보통 도함수 ∂_a

```
In[*]:= BD[ld, expr]
```

```
Out[*]:=
```

$$\partial_d R_{ac} + \partial_d R_{ab} F^b{}_c + \partial_d F^b{}_c R_{ab}$$

```
In[*]:= BD[ld, le, expr]
```

```
Out[*]:=
```

$$\partial_d \partial_e R_{ac} + \partial_d R_{ab} \partial_e F^b{}_c + \partial_d F^b{}_c \partial_e R_{ab} + \partial_d \partial_e R_{ab} F^b{}_c + \partial_d \partial_e F^b{}_c R_{ab}$$

Coordinate Basis에서 보통 도함수의 연속된 미분 $\partial_a \partial_b$ 는 교환된다:

```
In[*]:= BD[la, lb, MaxwellF[ua, ub]]
% // TindexSort
```

```
Out[*]:=
```

$$\partial_a \partial_b F^{ab}$$

```
Out[*]:=
```

$$0$$

공변 도함수 ∇_a

```
In[*]:= CD[ld, expr]
```

```
Out[*]:=
```

$$\nabla_d R_{ac} + \nabla_d R_{ab} F^b{}_c + \nabla_d F^b{}_c R_{ab}$$

계량 텐서 g_{ab} 는 공변 도함수 ∇_a 의 공변 상수이다:

```
In[*]:= Metricg[lb, lc]
CD[la, %]
```

```
Out[*]:=
```

$$g_{bc}$$

```
Out[*]:=
```

$$0$$

전자기장 F_{ab} 의 벡터 퍼텐셜 표현:

```
In[*]:= Tdefine[A[la]]
```

```
In[*]:= maxwellVectorPotentialRule =  
RuleUnique[MaxwellF[a_, b_], CD[a, A[b]] - CD[b, A[a]]]
```

```
Out[*]=  
Fa__ :=> DumFresh[ $\nabla_a A_b - \nabla_b A_a$ ]
```

```
In[*]:= MaxwellF[la, lb] /. maxwellVectorPotentialRule  
CDtoBD[%]  
% // TindexSort
```

```
Out[*]=  
 $\nabla_a A_b - \nabla_b A_a$ 
```

```
Out[*]=  
 $\partial_a A_b - \partial_b A_a - A_c \Gamma_{ab}^c + A_c \Gamma_{ba}^c$ 
```

```
Out[*]=  
 $\partial_a A_b - \partial_b A_a$ 
```

Maxwell 방정식:

```
In[*]:= CD[lb, MaxwellF[ua, ub]]
```

```
Out[*]=  
 $\nabla_b F^{ab}$ 
```

```
In[*]:= divF = % /. maxwellVectorPotentialRule  
CommuteCD[{ua, lb}, %]
```

```
Out[*]=  
 $\nabla_b \nabla^a A^b - \nabla_b \nabla^b A^a$ 
```

```
Out[*]=  
 $-\nabla_b \nabla^b A^a + \nabla^a \nabla_b A^b + A^b R^a_b$ 
```

```
In[*]:= lorentzGaugeRule = RuleUnique[CD[la_, A[ua_]], 0, PairIndexQ[la, ua]]
```

```
Out[*]=  
 $\nabla_{la} A_{ua} :=> DumFresh[0] /; PairIndexQ[la, ua]$ 
```

```
In[*]:= %% /. lorentzGaugeRule
```

```
Out[*]=  
 $-\nabla_b \nabla^b A^a + A^b R^a_b$ 
```

NB: No OrderCD[].

Lie 도함수 $\mathcal{L}_v$

```
In[*]:= Tdefine[v, 1]
```

```

In[*]:= expr
LD[v, expr]

Out[*]=

$$F^b_c R_{ab} + R_{ac}$$


Out[*]=

$$\mathcal{L}_v R_{ac} + \mathcal{L}_v R_{ab} F^b_c + \mathcal{L}_v F^b_c R_{ab}$$


$$\mathcal{L}_v q^a_b = v^p \partial_p q^a_b - (\partial_p v^a) q^p_b + (\partial_b v^p) q^a_p$$


In[*]:= Tdefine[q, "ab"]

In[*]:= LD[v, q[ua, lb]]
% // LDtoCD
% // CDtoBD
% // Tsimplify

Out[*]=

$$\mathcal{L}_v q^a_b$$


Out[*]=

$$\nabla_b v^c q^a_c - \nabla_c v^a q^c_b + \nabla_c q^a_b v^c$$


Out[*]=

$$\partial_b v^c q^a_c - \partial_c v^a q^c_b + \partial_c q^a_b v^c + \Gamma_{bc}^d q^a_d v^c - \Gamma_{cb}^d q^a_d v^c + \Gamma_{cd}^a q^d_b v^c - \Gamma_{cd}^a q^c_b v^d$$


Out[*]=

$$\partial_b v^c q^a_c - \partial_c v^a q^c_b + \partial_c q^a_b v^c$$


```

3.5 Creating Rules and Definitions

3.5.1 Simple Definition and Rules

```

In[*]:= Tdefine[tx, 2]; Tdefine[ty, 1]

mGRG에는 사용상의 위험 때문에 '전역적'인 변환 규칙을 사용하지 않는다.
인덱스 a와 b가 축약되었을 때 tx에 적용되는 '지역적'인 변환 규칙을 생성한다:

In[*]:= txTotyRule := tx[a_, b_] => ty[a] × ty[b] /; PairIndexQ[a, b]

지역적인 변환 규칙은 명시적으로 적용해야 한다:

In[*]:= tx[le, ue]
% /. txTotyRule

Out[*]=

$$tx_a^a$$


Out[*]=

$$ty_a ty^a$$


```



```

In[*]:= tx[lb, uc]
% /. txTotyRule

Out[*]=

$$tx_b^c$$


Out[*]=

$$tx_b^c$$


In[*]:= tx[le, ue] × tx[lb, uc]
% /. txTotyRule

Out[*]=

$$tx_a^a tx_b^c$$


Out[*]=

$$tx_b^c ty_a ty^a$$


In[*]:= tx[le, ue] × tx[lf, uf]
% /. txTotyRule

Out[*]=

$$tx_a^a tx_b^b$$


Out[*]=

$$ty_a ty_b ty^a ty^b$$


```

3.5.2 Pitfalls of Dummy Indices

변환 규칙에 축약된 인덱스가 포함되어 있으면 문제가 발생할 수 있다:

```

In[*]:= txTotyRule2 := tx[a_, b_] => ty[a] × ty[b] × ty[lc] × ty[uc] /; PairIndexQ[a, b]

In[*]:= tx[le, ue]
% /. txTotyRule2

Out[*]=

$$tx_a^a$$


Out[*]=

$$ty_a ty_b ty^a ty^b$$


In[*]:= tx[le, ue] × tx[lf, uf]
% /. txTotyRule2

Out[*]=

$$tx_a^a tx_b^b$$


Out[*]=

$$ty_a ty_b ty_c^2 ty^a ty^b ty^{c2}$$


```

이러한 문제는 축약된 인덱스를 더미 인덱스로 만드는 Dum 명령을 사용해서 일시적으로는 회피할 수 있다:

```
In[*]:= Dum[tx[le, ue] /. txTotyRule2] × Dum[tx[lf, uf] /. txTotyRule2]
```

```
Out[*]:= tya tyb tyc tyd tya tyb tyc tyd
```

또 다른 문제가 발생하는 상황을 알아 보자:

```
In[*]:= Tdefine[tv, 2]
```

```
In[*]:= tvTotxRule := tv[a_, b_] => 2 tx[a, b]
```

```
In[*]:= rule = {tvTotxRule, txTotyRule2}
```

```
Out[*]:= {tva_ => 2 txab, txa_ => tya tyb tyc tyc /; PairIndexQ[a, b]}
```

발생하는 문제를 명확히 보기 위해 문법을 확인해 보자:

```
In[*]:= On[SyntaxCheckFlag]
```

```
In[*]:= tv[la, ua]
% /. rule
```

```
Out[*]:= tvaa
```

```
Out[*]:= 2 tya tyb tya tyb
```

```
In[*]:= tv[la, ua] × tv[lb, ub]
% /. rule
```

```
Out[*]:= tvaa tvbb
```

Msg: non-scalar expression ty_c

Msg: non-scalar expression ty^c

```
Out[*]:= tya tyb tya tyb tyc2 tyc2
```

스칼라가 아닌 텐서를 제공하였다: (ty_c)²

```
In[*]:= scalarRule := ScalarCD[] => tx[la, lb] × tx[ua, ub]
```

```
In[*]:= ScalarCD[]2 /. scalarRule
```

```
Out[*]:= txab2 txab2
```

```
In[*]:= % // SyntaxCheck
```

```
Msg: non-scalar expression txab
```

```
Msg: non-scalar expression txab
```

```
Msg: non-scalar expression txab
```

```
General: Further output of Msg::err will be suppressed during this calculation.
```



```
Out[*]=
```

```
txab2 txab2
```

마찬가지로 스칼라가 아닌 텐서를 제공하였다: $(tx_{ab})^2$

기본 설정으로 복귀한다:

```
In[*]:= Off[SyntaxCheckFlag] (* default *)
```

3.5.3 Making Rules and Definitions Containing Dummy Indices

```
In[*]:= Tdefine[tx, 2]; Tdefine[tv, 2]; Tdefine[ty, 1]
```

변환 규칙에 축약된 인덱스가 있을 경우에도 사용할 수 있는 규칙을 생성한다:

```
In[*]:= txTotyRule =
```

```
RuleUnique[tx[la_, lb_], ty[la] × ty[lb] × ty[lc] × ty[uc], PairIndexQ[la, lb]]
```

```
Out[*]=
```

```
txla_lb → DumFresh[tya tyb tyLatin$26633 tyLatin$26633] /; PairIndexQ[la, lb]
```

```
In[*]:= expr1 = tx[lb, ub] × tx[lc, uc]
```

```
expr1 /. txTotyRule
```

```
Out[*]=
```

```
txaa txbb
```

```
Out[*]=
```

```
tya tyb tyc tyd tya tyb tyc tyd
```

```
In[*]:= tvTotxRule = RuleUnique[tv[la_, lb_], 2 tx[la, lb]]
```

```
Out[*]=
```

```
tvla_lb → DumFresh[2 txab]
```

```
In[*]:= rule := {txTotyRule, tvTotxRule}
```

```
In[*]:= tv[la, ua] × tv[lb, ub]
% /. rule
% /. rule
```

```
Out[*]=
```

$$tv_a^a tv_b^b$$

```
Out[*]=
```

$$4 tx_a^a tx_b^b$$

```
Out[*]=
```

$$4 ty_a ty_b ty_c ty_d ty^a ty^b ty^c ty^d$$

텐서 표현이 스칼라임을 명백히 표시하기 위해 Tscalar를 사용한다:

```
In[*]:= scalarRule = RuleUnique[ScalarCD[], tx[la, lb] × tx[ua, ub]]
```

```
Out[*]=
```

$$R \mapsto \text{DumFresh}[tx_{\text{Latin}\$26711\text{Latin}\$26713} tx^{\text{Latin}\$26711\text{Latin}\$26713}]$$

```
In[*]:= Tscalar[ScalarCD[]]^2
% /. scalarRule
```

```
Out[*]=
```

$$(R)^2$$

```
Out[*]=
```

$$(tx_{\text{Latin}\$26719\text{Latin}\$26721} tx^{\text{Latin}\$26719\text{Latin}\$26721})^2$$

3.5.4 Building Your Own Knowledge-Base Files

등각 변환을 위한 규칙 생성:

$$g_{ab} \rightarrow \Omega^2 g_{ab}, \quad g^{ab} \rightarrow \Omega^{-2} g^{ab}$$

```
In[*]:= confRule1 = RuleUnique[Metricg[la_, lb_],
  Ω^2 Metricg[la, lb], DnIndexQ[la] && DnIndexQ[lb]]
```

```
Out[*]=
```

$$g_{la_lb_} \mapsto \text{DumFresh}[\Omega^2 g_{ab}] /; \text{DnIndexQ}[la] \ \&\& \ \text{DnIndexQ}[lb]$$

```
In[*]:= confRule2 = RuleUnique[Metricg[ua_, ub_],
  Ω^-2 Metricg[ua, ub], UpIndexQ[ua] && UpIndexQ[ub]]
```

```
Out[*]=
```

$$g_{ua_ub_} \mapsto \text{DumFresh}\left[\frac{g^{ab}}{\Omega^2}\right] /; \text{UpIndexQ}[ua] \ \&\& \ \text{UpIndexQ}[ub]$$

```
In[*]:= confRule = {confRule1, confRule2};
```

```
In[*]:= Metricg[la, lb] /. confRule
```

```
Out[*]=
```

$$\Omega^2 g_{ab}$$

등각 변환에 따른 크리스토폴 심볼의 변환 공식:

$$\Gamma_{ab}^c \rightarrow \Gamma_{ab}^c + \delta_a^c \partial_b \ln \Omega + \delta_b^c \partial_a \ln \Omega - g_{ab} g^{cd} \partial_d \ln \Omega$$

```
In[*]:= GammaCD[la, lb, uc]
// GammaToMetric
/. confRule
```

```
Out[*]=
```

$$\Gamma_{ab}^c$$

```
Out[*]=
```

$$\frac{1}{2} \partial_a g_{bd} g^{cd} + \frac{1}{2} \partial_b g_{ad} g^{cd} - \frac{1}{2} \partial_d g_{ab} g^{cd}$$

```
Out[*]=
```

$$\frac{1}{2} \partial_a g_{bd} g^{cd} + \frac{1}{2} \partial_b g_{ad} g^{cd} - \frac{1}{2} \partial_d g_{ab} g^{cd} - \frac{\partial_d \Omega g_{ab} g^{cd}}{\Omega} + \frac{\partial_b \Omega g_{ad} g^{cd}}{\Omega} + \frac{\partial_a \Omega g_{bd} g^{cd}}{\Omega}$$

```
In[*]:= rhs = GammaCD[la, lb, uc] + %[[4]] + %[[5]] + %[[6]] // Absorbg
```

```
Out[*]=
```

$$\Gamma_{ab}^c + \frac{\partial_b \Omega \delta_a^c}{\Omega} + \frac{\partial_a \Omega \delta_b^c}{\Omega} - \frac{\partial^c \Omega g_{ab}}{\Omega}$$

```
In[*]:= affineConfRule = RuleUnique[GammaCD[la_, lb_, uc_],
rhs, DnIndexQ[la] && DnIndexQ[lb] && UpIndexQ[uc]]
```

```
Out[*]=
```

$$\Gamma_{\text{la_lb_uc_}} \mapsto \text{DumFresh} \left[\Gamma_{ab}^c + \frac{\partial_b \Omega \delta_a^c}{\Omega} + \frac{\partial_a \Omega \delta_b^c}{\Omega} - \frac{\partial^c \Omega g_{ab}}{\Omega} \right] /;$$

$$\text{DnIndexQ[la]} \&\& \text{DnIndexQ[lb]} \&\& \text{UpIndexQ[uc]}$$

```
In[*]:= GammaCD[la, lb, uc] /. affineConfRule
```

```
Out[*]=
```

$$\Gamma_{ab}^c + \frac{\partial_b \Omega \delta_a^c}{\Omega} + \frac{\partial_a \Omega \delta_b^c}{\Omega} - \frac{\partial^c \Omega g_{ab}}{\Omega}$$

Normal 좌표계에서

$$\partial_a g_{bc} = 0, \quad \Gamma_{ab}^c = 0$$

```
In[*]:= affineZeroRule := {BD[d_, GammaCD[a_, b_, c_]] \mapsto BD[d, GammaCD[a, b, c]],
GammaCD[a_, b_, c_] \mapsto 0};
affineZeroRule
```

```
Out[*]=
```

$$\{\hat{\partial}_a \Gamma_{a_c_} \mapsto \hat{\partial}_d \Gamma_{abc}, \Gamma_{a_c_} \mapsto 0\}$$

```
In[*]:= metricDerivZeroRule = BD[a_, Metricg[b_, c_]]  $\rightarrow$  0;
```

```
In[*]:= normalCoordRule = Flatten[{affineZeroRule, metricDerivZeroRule}];
```

```
In[*]:= {BD[la, Metricg[lb, lc]], GammaCD[la, lb, uc], BD[la, GammaCD[lb, lc, ud]]}
% /. normalCoordRule
```

```
Out[*]=
```

$$\{\partial_a g_{bc}, \Gamma_{ab}^c, \partial_a \Gamma_{bc}^d\}$$

```
Out[*]=
```

$$\{0, 0, \partial_a \Gamma_{bc}^d\}$$

등각 변환에 따른 리치 텐서의 변환 공식:

$$R_{ab} \rightarrow R_{ab} - [(n-2) \delta_a^c \delta_b^d + g_{ab} g^{cd}] \Omega^{-1} \nabla_c \nabla_d \Omega + [2(n-2) \delta_a^c \delta_b^d - (n-3) g_{ab} g^{cd}] \Omega^{-2} (\nabla_c \Omega) (\nabla_d \Omega)$$

```
In[*]:= expr1 = RicciCD[la, lb] // RiemannToGamma
```

```
Out[*]=
```

$$-\partial_a \Gamma_{cb}^c + \partial_c \Gamma_{ab}^c - \Gamma_{ac}^d \Gamma_{db}^c + \Gamma_{ab}^c \Gamma_{dc}^d$$

```
In[*]:= expr1 /. affineConfRule
```

```
Out[*]=
```

$$\begin{aligned} & -\partial_a \Gamma_{cb}^c - \frac{2 \partial_a \Omega \partial_b \Omega}{\Omega^2} + \frac{\partial_b \partial_a \Omega}{\Omega} + \partial_c \Gamma_{ab}^c + \frac{\partial_a g_{cb} \partial^c \Omega}{\Omega} - \frac{\partial_c g_{ab} \partial^c \Omega}{\Omega} - \frac{\partial_c \Omega \Gamma_{ab}^c}{\Omega} - \frac{\partial_b \Omega \Gamma_{ac}^c}{\Omega} + \\ & \frac{\partial_b \Omega \Gamma_{ca}^c}{\Omega} - \Gamma_{ac}^d \Gamma_{db}^c + \Gamma_{ab}^c \Gamma_{dc}^d - \frac{\partial_a \partial_b \Omega \delta^c_c}{\Omega} + \frac{2 \partial_a \Omega \partial_b \Omega \delta^c_c}{\Omega^2} + \frac{\partial_c \Omega \Gamma_{ab}^c \delta^d_d}{\Omega} - \frac{\partial_c \partial^c \Omega g_{ab}}{\Omega} + \\ & \frac{2 \partial_c \Omega \partial^c \Omega g_{ab}}{\Omega^2} - \frac{\partial_c \Omega \Gamma_{dc}^d g_{ab}}{\Omega} - \frac{\partial_c \Omega \partial^c \Omega \delta^d_d g_{ab}}{\Omega^2} + \frac{\partial_b \Omega \partial^c \Omega g_{ac}}{\Omega^2} + \frac{\partial^c \Omega \Gamma_{cb}^d g_{ad}}{\Omega} - \frac{\partial_b \Omega \partial^c \Omega g_{ca}}{\Omega^2} + \\ & \frac{\partial_a \partial^c \Omega g_{cb}}{\Omega} - \frac{\partial_a \Omega \partial^c \Omega g_{cb}}{\Omega^2} - \frac{\partial^c \Omega \partial^d \Omega g_{ad} g_{cb}}{\Omega^2} - \frac{\partial^c \Omega \Gamma_{ab}^d g_{cd}}{\Omega} + \frac{\partial^c \Omega \Gamma_{ac}^d g_{db}}{\Omega} + \frac{\partial^c \Omega \partial^d \Omega g_{ab} g_{dc}}{\Omega^2} \end{aligned}$$

```
In[*]:= % /. normalCoordRule
```

```
Out[*]=
```

$$\begin{aligned} & -\partial_a \Gamma_{cb}^c - \frac{2 \partial_a \Omega \partial_b \Omega}{\Omega^2} + \frac{\partial_b \partial_a \Omega}{\Omega} + \partial_c \Gamma_{ab}^c - \frac{\partial_a \partial_b \Omega \delta^c_c}{\Omega} + \\ & \frac{2 \partial_a \Omega \partial_b \Omega \delta^c_c}{\Omega^2} - \frac{\partial_c \partial^c \Omega g_{ab}}{\Omega} + \frac{2 \partial_c \Omega \partial^c \Omega g_{ab}}{\Omega^2} - \frac{\partial_c \Omega \partial^c \Omega \delta^d_d g_{ab}}{\Omega^2} + \frac{\partial_b \Omega \partial^c \Omega g_{ac}}{\Omega^2} - \\ & \frac{\partial_b \Omega \partial^c \Omega g_{ca}}{\Omega^2} + \frac{\partial_a \partial^c \Omega g_{cb}}{\Omega} - \frac{\partial_a \Omega \partial^c \Omega g_{cb}}{\Omega^2} - \frac{\partial^c \Omega \partial^d \Omega g_{ad} g_{cb}}{\Omega^2} + \frac{\partial^c \Omega \partial^d \Omega g_{ab} g_{dc}}{\Omega^2} \end{aligned}$$

In[*]:=

% // Absorbg

Out[*]=

$$\begin{aligned}
& -\partial_a \Gamma_{cb}^c - \frac{4 \partial_a \Omega \partial_b \Omega}{\Omega^2} + \frac{\partial_b \partial_a \Omega}{\Omega} + \partial_c \Gamma_{ab}^c - \frac{\partial_a \partial_b \Omega \delta^c_c}{\Omega} + \\
& \frac{2 \partial_a \Omega \partial_b \Omega \delta^c_c}{\Omega^2} - \frac{\partial_c \partial^c \Omega g_{ab}}{\Omega} + \frac{3 \partial_c \Omega \partial^c \Omega g_{ab}}{\Omega^2} - \frac{\partial_c \Omega \partial^c \Omega \delta^d_d g_{ab}}{\Omega^2} + \frac{\partial_a \partial^c \Omega g_{cb}}{\Omega}
\end{aligned}$$

이하 생략...

3.6 The Riemann and Related Rules

3.6.1 Organization of Rules

3.6.2 Automatic Riemann Rules

3.6.3 Riemann Rules Used with the ApplyRules Functions

In[*]:=

```
riemannRule7 :=
RuleUnique[RiemannCD[a_, b_, c_, d_] × RiemannCD[e_, f_, g_, h_],
  -1/2 RiemannCD[li, lj, lk, d] × RiemannCD[e, uk, ui, uj],
  PairIndexQ[a, f] && PairIndexQ[b, g] && PairIndexQ[c, h]]
```

In[*]:=

```
RiemannCD[la, lb, lc, ld] × RiemannCD[le, ua, ub, uc]
% /. riemannRule7
```

Out[*]=

$$R_{abcd} R_e^{abc}$$

Out[*]=

$$-\frac{1}{2} R_{bcad} R_e^{abc}$$

나머지 생략...

3.6.4 Rules Used individually

3.6.5 Conversion Rules

Christoffel 심볼의 계량 텐서 표현:

```
In[*]:= GammaCD[la, lb, uc]
% // GammaToMetric
```

```
Out[*]=
```

$$\Gamma_{ab}^c$$

```
Out[*]=
```

$$\frac{1}{2} \partial_a g_{bd} g^{cd} + \frac{1}{2} \partial_b g_{ad} g^{cd} - \frac{1}{2} \partial_d g_{ab} g^{cd}$$

리만 텐서의 Christoffel 심볼 표현:

```
In[*]:= RiemannCD[la, lb, lc, ld]
% // RiemannToGamma
```

```
Out[*]=
```

$$R_{abcd}$$

```
Out[*]=
```

$$-\Gamma_{aed} \Gamma_{bc}^e + \Gamma_{ac}^e \Gamma_{bed} - \partial_a \Gamma_{bc}^e g_{de} + \partial_b \Gamma_{ac}^e g_{de}$$

```
In[*]:= RicciCD[la, lb]
% // RiemannToGamma
```

```
Out[*]=
```

$$R_{ab}$$

```
Out[*]=
```

$$-\partial_a \Gamma_{cb}^c + \partial_c \Gamma_{ab}^c - \Gamma_{ac}^d \Gamma_{db}^c + \Gamma_{ab}^c \Gamma_{dc}^d$$

```
In[*]:= ScalarCD[]
% // RiemannToGamma
```

```
Out[*]=
```

$$R$$

```
Out[*]=
```

$$\Gamma_a^{ab} \Gamma_{cb}^c - \Gamma_{ab}^c \Gamma_c^{ab} - \partial_a \Gamma_{bc}^b g^{ac} + \partial_a \Gamma_{bc}^a g^{bc}$$

Trace-Free 리치 텐서와 아인슈타인 텐서:

```
In[*]:= traceFreeRicci =
RicciCD[la, lb] -  $\frac{1}{\text{GetDimension[Latin]}}$  Metricg[la, lb] × ScalarCD[]
```

```
Out[*]=
```

$$R_{ab} - \frac{1}{3} g_{ab} R$$


```
In[*]:= traceFreeRicci Metricg[ua, ub]
% // Absorbg
% /. Kdelta[ua, la] → GetDimension[Latin]
```

Out[*]=

$$g^{ab} R_{ab} - \frac{1}{3} g_{ab} g^{ab} R$$

Out[*]=

$$R - \frac{1}{3} \delta^a_a R$$

Out[*]=

$$0$$

```
In[*]:= einsteinG[a_, b_] := RicciCD[a, b] - \frac{1}{2} Metricg[a, b] \times ScalarCD[]
einsteinG[la, lb]
```

Out[*]=

$$R_{ab} - \frac{1}{2} g_{ab} R$$

Christoffel 심볼 미분의 특별한 성질:

```
In[*]:= BD[la, GammaCD[lb, lc, uc]] - BD[lb, GammaCD[la, lc, uc]]
% // GammaToMetric // Tsimplify
```

Out[*]=

$$\partial_a \Gamma_{bc}^c - \partial_b \Gamma_{ac}^c$$

Out[*]=

$$\frac{1}{2} \partial_a g^{cd} \partial_b g_{cd} - \frac{1}{2} \partial_a g_{cd} \partial_b g^{cd}$$

```
In[*]:= % /. BDinvgRule[]
% // Tsimplify
```

Out[*]=

$$\frac{1}{2} \partial_a g_{cd} \partial_b g_{ef} g^{ce} g^{df} - \frac{1}{2} \partial_a g_{cd} \partial_b g_{ef} g^{ec} g^{fd}$$

Out[*]=

$$0$$

```
In[*]:= affineDRule = RuleUnique[BD[la_, GammaCD[lb_, lc_, uc_]],
BD[lb, GammaCD[la, lc, uc]], PairIndexQ[lc, uc] && ! IndexOrderedQ[{la, lb}]]
```

Out[*]=

$$\hat{\partial}_{\text{la}_\Gamma \text{lb}_{\text{lc}} \text{uc}} \mapsto \text{DumFresh}[\partial_b \Gamma_{a \text{Latin\$27762}}^{\text{Latin\$27762}}] /; \text{PairIndexQ}[lc, uc] \&\& ! \text{IndexOrderedQ}[\{la, lb\}]$$

```
In[*]:= BD[lb, GammaCD[la, lc, uc]] /. affineDRule
Out[*]=
```

$$\partial_a \Gamma_{bc}^c$$

3.7 Using Contexts with MathTensor

직교 좌표계에서 계량이 상수인 시공간을 설정한다:

```
In[*]:= SetConstantMetric[]
```

```
In[*]:= BD[la, Metricg[lb, lc]]
Out[*]=
```

0

계량 텐서를 설정한다:

```
In[*]:= SetConstantMetric[{-1, 1, 1}]
```

```
In[*]:= Table[Metricg[-i, -j], {i, 3}, {j, 3}] // MatrixForm
Epsilon[-1, -2, -3]
```

```
Out[*]//MatrixForm=
```

$$\begin{pmatrix} -1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

```
Out[*]=
```

1